erdealer — Project Onboarding Guide

React (CRA, React 19 + react-router-dom v7) single-page app for Vedika Finance's e-Rickshaw dealer loan channel.

One codebase serves three different user roles — **Dealer**, **FI** (Field Investigation officer), and **HO** (Head Office) — with role-based routing and a shared customer/application record that flows through every stage from dealer onboarding to loan disbursement.

No backend code lives in this repo — it's a pure frontend calling several external REST APIs (see the API integration map below).

Getting started

```
npm install
npm start      # runs on https://localhost:3000 (HTTPS=true is set in package.json)
```

There is no `.env` — API base URLs are hardcoded in `src/component/api/apiRoutes.js`. Swap the commented `BASE_URL` lines there to point at local vs. UAT vs. production.

Tech stack

- **React 19**, **react-router-dom v7**, bootstrapped with `react-scripts` (CRA) — no TypeScript, no state-management library (component-local `useState` / `useEffect` only).
- No CSS framework — one hand-written stylesheet per page under `src/assets/css/`, several of which are shared across multiple components.
- No PDF/print library — where a "generate PDF" feature exists (Final Review), it uses the browser's native print-to-PDF (`window.print()` + `@media print` CSS), not a package like `jspdf`.
- Auth/session state is just `localStorage` — no cookies, no JWT-in-memory store.

Folder structure

```
src/
  component/
    api/apiRoutes.js              — every backend endpoint URL, in one place
    utils/
      leadStatus.js                — the shared post-sanction status machine
      DealerRoute.jsx              — route guard for Customer Onboarding pages
    CustomerOnboarding/            — the 3-stage form the DEALER fills in per lead
      ApplicantVerification.jsx    — Stage 1: applicant + co-applicant KYC/CB
      DocumentCollection.jsx       — Stage 2: documents, bank, references, quotation
      OfferSummary.jsx             — Stage 3: loan structure + LTV/EMI + accept
    FI/                            — pages used by FI officers and HO (shared folder)
      FIDashboard.jsx, CustomerDetails.jsx, SanctionReview.jsx, FinalReview.jsx
    Login.jsx, OtpVerify.jsx, DealerOnboardingForm.jsx, DealerDashboard.jsx,
    DealerDetails.jsx, ApplicationStatus.jsx, SourcingPipeline.jsx, LeadDetails.jsx,
    HoDashboard.jsx, HODeliveryVerification.jsx, LoanExecution.jsx, VehicleDelivery.jsx
  assets/css/                      — one stylesheet per page (imported directly, not modules)
  App.js                           — all routes
```

Note: pages that look role-specific by name aren't always folder-scoped consistently — e.g. `HoDashboard.jsx` and `HODeliveryVerification.jsx` live at the top level of `component/`, not in a `HO/` folder, while `SanctionReview.jsx` / `FinalReview.jsx` (also HO-facing) live under `FI/`. Don't assume folder = role; check `App.js` for what a route actually renders.

User roles

Role	How they log in	Landing page	What they do
Dealer	`Login.jsx` → OTP (`OtpVerify.jsx`)	`/dealer-dashboard`	Sources leads, fills the 3-stage Customer Onboarding form, adds vehicle delivery details, completes e-Mandate/e-Sign
FI	Same login, `role === "FI"`	`/fi-dashboard`	Conducts field investigation, runs Video PD, submits FI data
HO	Same login, `role === "HO"`	`/ho-dashboard`	Approves/declines sanctions, verifies vehicle delivery, does final review + disbursement

Role is read from `localStorage.getItem("user")` (JSON), specifically the `role` field. `DealerRoute.jsx` is the only route guard in the app — it protects the 3 Customer Onboarding pages so only a logged-in Dealer can reach them (`Navigate to="/"` otherwise). FI/HO pages currently have **no route guard** — they're reachable by URL by anyone with a `customerId`, relying on the fact that nobody outside those teams knows the URLs.

Session & auth

- Login (`Login.jsx`) hits `API.LOGIN` , stores the response in `localStorage.user` and `localStorage.dealer_status` , then sends an OTP and redirects to `/otp-verify` .
- `OtpVerify.jsx` verifies via `API.OTP_VERIFY` (`verify-mobile-otp`). The response's `data.id` is the dealer's real primary key — it's explicitly remapped to `dealer_id` when merging into `localStorage.user` , because **every other page in the app reads** `dealerData.dealer_id` , not `.id` . (This exact mismatch caused a real bug once — see the "Known gaps" section.)
- There is no token refresh — `localStorage.getItem("token")` is sent as `Authorization: Bearer <token>` on several confirmed endpoints (sanction save, vehicle delivery, FI details). If the token expires there's no re-auth flow; the user would need to log in again.

The core data object: the customer/application record

Nearly every page in FI/HO fetches the same record:

```
GET {baseURL}/public/api/customer/{customerId}
```

This single record accumulates fields from every stage of the funnel — applicant/co-applicant KYC, vehicle/loan terms, bank details, references, dealer info, uploaded document paths, CB responses, FI/Video-PD fields, sanction figures, and status flags. There's no formal schema doc for it; the closest thing is **`FinalReview.jsx`** , which reads and displays essentially every field this record can have, organized under 10 headings (Applicant, Co-Applicant, Reference, Bank, Dealer, Vehicle, Vehicle Identification, FI, Onboarding Document, Delivery Documents).

Key nested/encoded fields (parsed the same way in `LeadDetails.jsx` , `FinalReview.jsx` , and `DealerDetails.jsx` via a small `parseStoredJson` helper that unwraps a value that may be double-JSON-encoded):

- `applicant_cb_response` / `co_applicant_cb_response` — raw CIBIL/CB response (`cb_score` , `odAmount` , `dpd` , `outstandingBalanceAmount` , `cbHitStatus` , `cbCheckStatus` , etc.)
- `penny_drop_response` — raw bank penny-drop API response (`data.beneficiary_name_with_bank` , `data.verified`)

Status flags that drive the funnel

Field	Meaning
`fi_status`	0/missing = FI not done; truthy (1 or 3 = Video PD Completed) = FI done
`sanction_status`	1 = HO approved the sanction
`vehicle_delevery_status` (sic — genuinely misspelled in the API)	0/missing = pending; 1 = dealer submitted vehicle, awaiting HO approval; 2 = HO approved
`mandate_status`	tracked/gated inside `LoanExecution.jsx` itself, not part of the shared status machine

---

The status machine — `src/component/utils/leadStatus.js`

`getStatusInfo(customer)` is the single source of truth for "what stage is this lead at", used by `SourcingPipeline.jsx`, `HoDashboard.jsx`, `FIDashboard.jsx`, `DealerDashboard.jsx`, and `LeadDetails.jsx` to pick which action button to show and which status badge to render.

```
fi_status falsy                        → fi_pending            "FI Pending"
sanction_status != 1                   → sanction_pending      "Sanction Pending"
sanction_status == 1, delivery 0/none  → post_sanction_pending "Post Sanction Pending"
sanction_status == 1, delivery == 1    → delivered             "Vehicle Added - Pending HO Approval"
sanction_status == 1, delivery == 2    → mandate_approved       "Mandate Approved"  (funnel complete)
```

Important, non-obvious design decision: post-sanction, **e-Mandate + e-Sign now happens BEFORE vehicle delivery**, not after (this was flipped mid-project — see git history around `SourcingPipeline.jsx` / `LoanExecution.jsx` / `VehicleDelivery.jsx`). Both the mandate/eSign step and the add-vehicle step are shown as **one single** `post_sanction_pending` stage in every dashboard — there's no separate visible stage for "mandate pending" anymore. The single "Add vehicle details" button dealers see for this stage actually opens **Loan Execution** first (`/loan-execution/:id`); only once mandate + e-Sign are both done there does it hand off to `/vehicle-delivery/:id`.

Page-by-page route map (`src/App.js`)

Route	Component	Who uses it
`/`	`Login.jsx`	Everyone (login + "New Join" dealer signup flow)
`/dealer-onboarding`	`DealerOnboardingForm.jsx`	New dealer self-registration
`/otp-verify`	`OtpVerify.jsx`	Post-login OTP step
`/dealer-dashboard`	`DealerDashboard.jsx`	Dealer home
`/ho-dashboard`	`HoDashboard.jsx`	HO home — dealer list + customer list w/ action buttons
`/sourcing-pipeline`	`SourcingPipeline.jsx`	Dealer's lead list w/ stage-aware action buttons
`/lead-details/:customerId`	`LeadDetails.jsx`	Dealer's read-only full lead view
`/application-status`	`ApplicationStatus.jsx`	Dealer's own onboarding-application status (re: their dealership, not a customer lead)
`/applicant-verification` (DealerRoute-guarded)	`ApplicantVerification.jsx`	Customer Onboarding Stage 1
`/document-collection` (DealerRoute-guarded)	`DocumentCollection.jsx`	Customer Onboarding Stage 2
`/offer-summary` (DealerRoute-guarded)	`OfferSummary.jsx`	Customer Onboarding Stage 3
`/sanction-review/:customerId?`	`FI/SanctionReview.jsx`	HO — approve/decline/refer-back sanction, edit loan structure, view FI/Video-PD detail
`/vehicle-delivery/:customerId?`	`VehicleDelivery.jsx`	Dealer — enter vehicle ID + upload delivery docs
`/dealer-details/:dealerId`	`DealerDetails.jsx`	HO — review a dealer's onboarding application
`/fi-dashboard`	`FI/FIDashboard.jsx`	FI officer's customer list
`/customer-details/:customerId`	`FI/CustomerDetails.jsx`	FI officer — fill in FI form, run Video PD
`/loan-execution/:customerId?`	`LoanExecution.jsx`	Dealer — e-Mandate + e-Sign (now the FIRST post-sanction step)
`/ho-delivery-verification/:customerId?`	`HODeliveryVerification.jsx`	HO — approve/refer-back the delivered vehicle
`/final-review/:customerId?`	`FI/FinalReview.jsx`	HO — consolidated read-only view of the ENTIRE customer file + disbursement trigger

End-to-end flow

```
Dealer Login → OTP verify
  → Dealer Dashboard → "Start new lead" → Applicant Verification (Stage 1: KYC + CB)
  → Document Collection (Stage 2: docs, bank+penny-drop, references, quotation)
  → Offer Summary (Stage 3: loan structure, LTV/EMI, accept terms)
      ↓
FI officer picks it up (FI Dashboard → Customer Details)
  → fills FI form, runs Video PD (Think360) → fi_status becomes truthy
      ↓
HO reviews (Sourcing Pipeline / HO Dashboard → Sanction Review)
  → approve / decline / refer back, can revise loan amount/tenure/on-road/ex-showroom
    (LTV/EMI auto-recalculate) → sanction_status = 1
      ↓
Dealer: "Add vehicle details" → actually opens Loan Execution FIRST
  → e-NACH mandate + e-Sign (Aadhaar OTP) → on completion, forwarded to Vehicle Delivery
  → enters vehicle/engine/chassis/battery numbers + uploads delivery docs
    (delivery photo, invoice, insurance, RC application, quotation*, disclaimer†,
    RC challan†, battery photo†, NO HPA letter†, Form 22†)
      ↓
HO Delivery Verification → approve/refer-back → vehicle_delevery_status = 2
      ↓
HO Final Review (consolidated view of everything above) → "Ready To Disbursement"
  (currently: generates a print-to-PDF snapshot of the whole file; see Known gaps)
```

* Quotation at vehicle-delivery auto-fills from the one already uploaded during Customer Onboarding (`customer.quotation_image`) — no need to re-upload unless replacing it.

† These 6 delivery documents were added later and their backend field names are **unconfirmed** (see Known gaps).

API integration map

All endpoints live in `src/component/api/apiRoutes.js` as a flat `API` object. Two different backend hosts are involved:

- `BASE_URL` (`vfpl.teamvedika.com/dealer-api/public/api`) — the main dealer-app backend (login, dealer/customer CRUD, sanction, vehicle delivery, FI, penny-drop).
- `BASE_URL_CUSTOMER` (`weblos.teamvedika.com/api/sweb/v1`) — third-party KYC/CB provider hit directly from the browser during Customer Onboarding (OTP, eKYC, PAN verify, CB check).

Confirmed via curl (safe to rely on)

API key	Method / path	Notes
`SEND_OTP`	POST `/send-otp`	`{ mobile }`
`VERIFY_MOBILE_OTP` / `OTP_VERIFY`	POST `/verify-mobile-otp`	multipart `{ mobile, otp, dealer_id }`
`FI_DETAILS_GET_API`	GET `/er-loan/fi/<cust_id>`	Full FI submission record: `request_payload` + `response_payload` (Video PD webhook, once completed)
`SANCTION_SAVE_API`	POST `/er-loan/save-sanction`	`{ cust_id, kyc_id, emp_id, decision, sanction_amt, sanction_note }`
`UPDATE_LOAN_STRUCTURE_API`	POST `/er-loan/update-sanction-details`	`{ cust_id, kyc_id, sanction_amt, tenure, on_road_price, ex_showroom_price, ltv, emi, sanction_note }`
`VEHICLE_DELIVERY_SAVE_API`	POST `/er-loan/save-vehicle-delivery`	multipart, `{ cust_id, kyc_id, emp_id, vehicle_no, engine_no, chassis_no, battery_no, battery_make }` + document files
`VEHICLE_DELIVERY_GET_API`	GET `/er-loan/get-vehicle-delivery?cust_id=`	Returns vehicle ID fields + confirmed 4 document paths
`HO_DELIVERY_VERIFICATION_SAVE_API`	POST `/er-loan/save-ho-delivery-verification`	`{ cust_id, verified_by, remarks, action, vehicle_checks[], evidence_checks[] }`
`CHECK_MANDATE_STATUS_API`	POST `/er-loan/check-mandate-status`	`{ cust_id, kyc_id }`
`INITIATE_ESIGN_API`	POST `/er-loan/initiate-esign` (x-www-form-urlencoded)	`{ kyc_id, emp_id }`

Unconfirmed / placeholder — do not assume these work

API key	Where used	Status
`AUTO_DISBURSEMENT_API` (`/er-loan/auto-disbursement`)	`FinalReview.jsx`	**Made up**, following the `er-loan/*` naming convention. Not called from the current disbursement flow — see Known gaps.
6 delivery document field names: `quotation` , `disclaimer` , `rc_challan` , `battery_photo` , `no_hpa_letter` , `form_22`	`VehicleDelivery.jsx` upload, `VEHICLE_DELIVERY_GET_API` read-back	Guessed snake_case names — only the original 4 (`delivery_photo` , `invoice` , `insurance` , `rc_application`) were curl-confirmed
`applicant_gender` , `applicant_marital_status` , `co_applicant_relation` , `co_applicant_gender` , `co_applicant_marital_status` , `occupation`	`ApplicantVerification.jsx` / `DocumentCollection.jsx` save payloads	Sent to the backend but never explicitly confirmed the column names match

---

Business rules worth knowing (non-obvious, easy to accidentally revert)

- **Unmarried female applicant → not eligible.** `ApplicantVerification.jsx` blocks proceeding (button disabled + inline message) whenever `gender === "female" && maritalStatus === "unmarried"` .
- **Co-applicant relation options depend on the applicant's gender + marital status** (`getCoApplicantRelationOptions` in `ApplicantVerification.jsx`): unmarried → only "Father"; married female → Husband/Son/Father-in-law; married male → Wife/Mother/Daughter-in-law.
- **Applicant, co-applicant, and both references must all have distinct mobile numbers** — enforced in `DocumentCollection.jsx` (`findDuplicateMobile`) against values persisted from Stage 1 via `localStorage` .
- **CB dedupe reject reasons** are decoded from `cbCheckStatus` codes (2, 21–30) into human-readable causes (Aadhaar/mobile/bank-account duplicate) in `ApplicantVerification.jsx` 's `getCbRejectReason` — the raw API message is a useless generic string ("CB Check Failed - cbCheckStatus: 25").
- **LTV/EMI use a fixed 30% annual policy rate**, calculated identically in `OfferSummary.jsx` and `SanctionReview.jsx` (`INTEREST_RATE_ANNUAL = 30`) — if the policy rate ever changes, update both places.
- **Video PD location is authoritative over browser GPS.** `CustomerDetails.jsx` (FI officer's page) used to also capture the officer's own device GPS (`pdLatitude` / `pdLongitude` via `navigator.geolocation`) as a fallback distance source; this was removed entirely — distance-from-branch / distance-to-customer now only compute from `vpdLocation` , the location embedded in the Video PD provider's own webhook response.

Known gaps / TODOs

1. `AUTO_DISBURSEMENT_API` **is not a real endpoint.** `FinalReview.jsx`'s disbursement button currently just generates a print-to-PDF snapshot of the page and marks the lead "Disbursed" locally — it does not call any backend. Needs the real endpoint (URL, method, request/response shape) before this can actually disburse anything.
2. **6 of the 10 vehicle-delivery document field names are unconfirmed** (see API table above) — if uploads silently aren't being saved, this is the first thing to check.
3. **No route guard on FI/HO pages** — anyone with a `customerId` can hit `/sanction-review/:id`, `/final-review/:id`, etc. directly. Only the 3 Customer Onboarding pages are guarded (`DealerRoute.jsx`).
4. **No token refresh flow** — an expired `localStorage.token` just causes silent 401s on the Bearer-authenticated endpoints; there's no re-auth prompt.
5. **Print-to-PDF, not a real PDF file** — "Generate PDF" on Final Review relies on the browser's print dialog ("Save as PDF"); it doesn't produce a downloadable `.pdf` file directly. No PDF library is installed in this project.

Where to look first for common tasks

Task	Start here
Add a field to a stage of Customer Onboarding	The relevant stage file under `CustomerOnboarding/`, then mirror the display in `FinalReview.jsx` / `LeadDetails.jsx` / `FI/CustomerDetails.jsx`
Change what triggers a status badge/action button	`src/component/utils/leadStatus.js`, then check every consumer (`SourcingPipeline.jsx`, `HoDashboard.jsx`, `FIDashboard.jsx`, `LeadDetails.jsx`)
Add/rename a backend endpoint	`src/component/api/apiRoutes.js` — always note "Confirmed via curl" or explicitly mark as a guess
Change the LTV/EMI formula	Duplicated in `OfferSummary.jsx` and `SanctionReview.jsx` — update both
Add a new document upload	Follow the pattern in `VehicleDelivery.jsx` (`DELIVERY_DOC_FIELD` map + `handleFileUpload` switch + FormData append)